



BIO - Zvýraznění krevního řečiště v prstu

Project documentation

Adam Kučík (xkucik00)

Kryštof Paulík (xpauli08)

November 11, 2025

Abstract

During the past two decades, automatic identification and verification of an individual have become increasingly common. Although biometric methods such as fingerprint or facial recognition are widely used, they have certain security vulnerabilities, as they can be easily spoofed. One of the alternatives to this used approach could be finger-vein recognition due to high security and reliability. The finger veins are obtained by NIR sensors to capture the unique vein pattern beneath the skin, which has also been proven to be distinctive for each individual and difficult to spoof.

1 Introduction

Finger-vein recognition faces the challenge of low visibility of the vein in images captured by NIR sensors. The issue is commonly solved by the recent advances and growing popularity of neural networks.

In this work, we compare the *Maximum Curvature* method for finger-vein detection on raw finger-vein images with its performance on enhanced images. The enhancement process combines a convolutional neural network with the additional use of Gabor filters to improve vein visibility.

2 Dataset

In this work, we had available 3 datasets: MMCBN_6000 [1], FV-USM¹ and the Finger Vein Database². All images were converted to JPG format and cropped to a uniform size of 320×240 pixels. Initially, we intended to use all available datasets to maximize the amount of training data. However, we were unable to achieve reasonable results in finger-vein enhancement, as discussed in detail in Section 3. Therefore, for our experiments, we selected the MMCBNU_6000 dataset, which contained enough data for our experiments.

3 Finger vein enhancement

In this section, we discuss the implementation of our finger-vein image enhancer, which combines a convolutional neural network with Gabor filters. At first, we focused on finding suitable parameters for the Gabor filters to achieve results that would slightly enhance the visibility of finger-veins, before integrating the neural network. Before detailing the parameter selection process, let us briefly review the formulation of the Gabor filter and the role of its main parameters.

A 2D Gabor filter is defined as [2]:

$$G(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi \frac{x'}{\lambda} + \psi\right), \quad (1)$$

where $G(x, y)$ represents the filter kernel applied at spatial coordinates (x, y) . The variables x' and y' correspond to the coordinates of the point after rotation by an angle θ , and are given by:

$$x' = x \cos \theta + y \sin \theta, \quad (2)$$

$$y' = -x \sin \theta + y \cos \theta. \quad (3)$$

The parameters of the Gabor filter have the following meanings [2]:

- σ is the standard deviation of the Gaussian envelope,
- λ is the wavelength of the sinusoidal,

¹<https://www.kaggle.com/datasets/ryeltsin/bdfvum>

²<https://www.kaggle.com/datasets/ryeltsin/finger-vein>

- γ is the spatial aspect ratio,
- ψ is the phase offset,
- θ specifies the orientation of the filter.

Gabor filters also include another important parameter, k_{size} (Gabor convolution size), which defines the spatial size of the filter kernel and determines how large an area of the image is used for convolution. After experimenting with different configurations, we chose the following parameters for our initial Gabor filter: $k_{size} = 61$, $\sigma = 5$, $\lambda = 16.0$, $\gamma = 1$, $\psi = 0$ and orientations in degrees = $[0, 18, 65, 90, 115, 160]$, these parameters are taken from [2].

With the Gabor filter implementation completed, we extended the process by integrating it with a convolutional neural network. We used a small convolutional neural network consisting of three convolutional layers, each followed by a ReLU activation function.

A semi-supervised training approach was used. For the supervised part, we manually segmented finger-vein lines using white lines drawn directly on the raw images. As we had 3 datasets, for each we made 20 images with corresponding segmented masks, so a total of 60 images for supervised training. White lines were added to emphasize the finger-vein structures for the neural network to focus on the vein pattern. For unsupervised training, we used 14 000 images and for testing about 1 500 images.

Based on research articles in this field, we were able to find the structure of the convolutional neural network. However, the exact loss function used was not specified, which led us to experiment and identify the most effective one. The initial loss function was selected as the *Mean Squared Error* for the supervised training and the *smooth loss* function for the unsupervised training. The results can be seen in the following images 1:

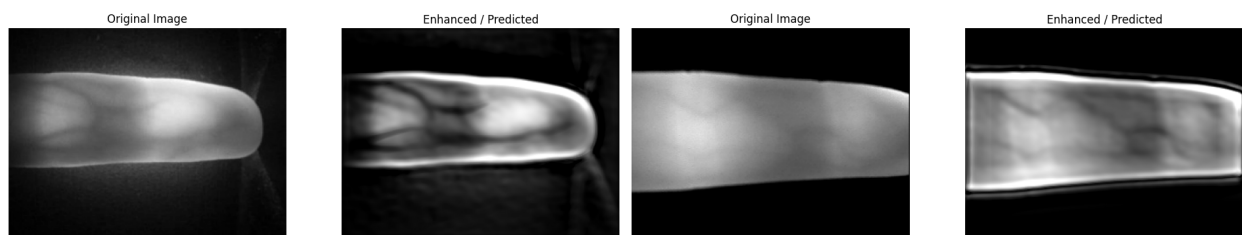


Figure 1: Comparison between the original image and the enhanced result. The left image is from the FV-USM dataset and the right image is from the MMCBNU_6000 dataset.

The model was able to enhance the finger-vein patterns, making them more distinguishable to the human eye compared to the original images. The problem was that it was heavily focused on the outer line of the finger and also started to create artifacts outside of the finger region.

We decided to rework our masks to be only black background with the white lines corresponding to the vein locations. This approach indeed helped with the focus of Gabor filter on the outside of the finger, but the model was starting to produce only black images on the output. This occurred because supervised training encouraged the network to generate mostly black images with only faint white vein lines, while unsupervised training reinforced the tendency toward fully black images.

At this stage, we decided to use only the MMCBNU_6000 dataset, as we found that each dataset benefited from slightly different configurations of the Gabor filter parameters, as noted in [3]. Initially, the results were similar to those obtained in previous experiments. Therefore, after further testing, we replaced the smooth loss function with an *Edge-Preserving Loss*, which encouraged the model to maintain continuous vein lines.

To further improve the results, we expanded our set of segmentation masks. Additional masks were generated using the *Maximum Curvature* method provided by the *biosig* library enhancer [4], more in section 4, which helped to capture more vein structures in the images. Moreover, we manually created an additional

20 segmented images. These were combined with automatically generated masks, resulting in a total of 300 annotated images, of which 40 were manually segmented.

After experimenting with various dataset sizes for unsupervised learning, we achieved the best performance using 800 images for the unsupervised training set. The model was then trained for 10 epochs. For the final evaluation we kept 500 images.

Once we obtained a reasonably well-performing model using fixed Gabor filters, we extended our approach to an adaptive Gabor filter setup, in which the Gabor parameters were updated during training. We tried two different configurations for the adaptive Gabor filter. The first configuration used the same structure as the fixed model, but the filter parameters were made learnable. The second configuration also retained the same base structure but increased the number of Gabor filters from six to ten. The larger model was inspired by [5]. The tables with learned Gabor parameters can be found in the Appendix A.

The results of the final enhancer can be seen in the following figure 2:

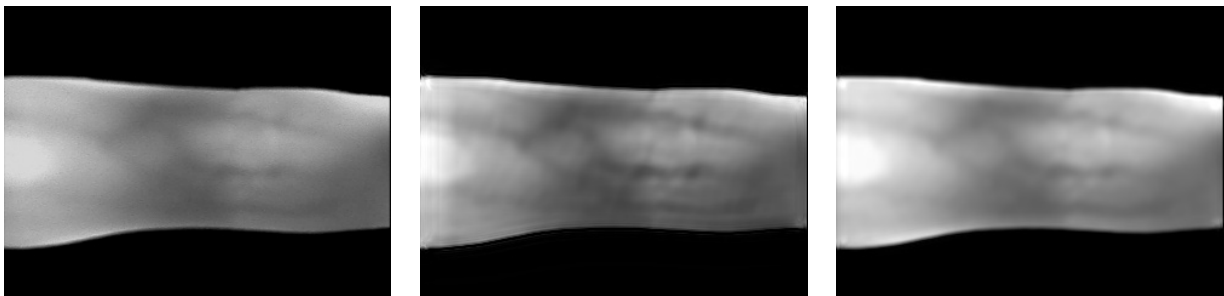


Figure 2: The left image shows the original input. The middle image presents the result from the enhancer using fixed Gabor parameters, while the right image shows the result obtained with adaptive Gabor parameters using 10 orientations.

In the following section, we will discuss the comparison results of the *maximum curvature* method on both the original and enhanced images.

4 Comparison

For extracting veins, we used the *Maximum Curvature* method, implemented through an extractor from the `bob.bio.vein` library with a Gaussian smoothing parameter of $\sigma = 4.8$, the value was chosen from a set of values which we tested against the baseline mask and visually the extraction fitted the baseline mask the best. The MC method is a vein enhancement technique designed to highlight line-shaped, valley-like structures such as finger veins, while reducing background texture and noise.

The algorithm analyzes the second directional derivatives of image intensity in multiple orientations and identifies the direction of maximum negative curvature, which corresponds to the darker regions (veins) on a brighter background.

In our implementation, each grayscale image is first restricted to the finger only using a mask generated by a modified Lee adaptive filter [6]. The MC extractor is then applied only within this mask to avoid false detections in the background. The result is binarized using Otsu’s thresholding onto a binary map, a black image, where white pixels represent veins.

As a baseline for comparison, as well as for training the neural net, we used finger-vein masks created by the *biosig* tool [4]. The tool has its own *conda* environment with commands for enhancing the image or extracting veins from it using Maximum Curvature.

We wrote a shell script `process_images.sh` that took a batch of images we dedicated for testing, enhanced those images using *biosig* and then extracted the finger-vein masks using **biosig**’s own implementation of Maximum Curvature method. The output of this script is shown in Figure 3.

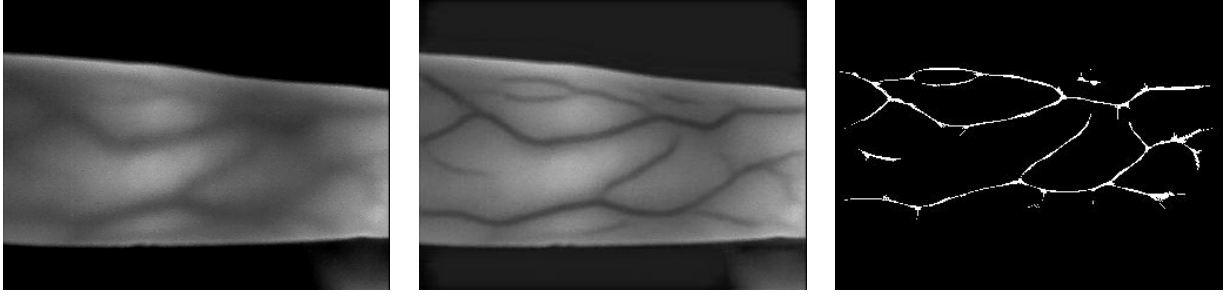


Figure 3: Input of process_image.sh (left); Enhanced image using biosig (middle); Detected veins (right)

However, we later found out that the vein masks generated by *biosig* did not perfectly fit "over" the corresponding original or even enhanced images. This occurred because *biosig* slightly modifies the image dimensions and even rotates the image so that the veins are vertical, probably during preprocessing. At first we thought that this modification is consistent and could be fixed with cropping around the center of the image, but we soon realized that this was not the case. Since accurate pixel correspondence is critical for evaluating and comparing the methods, we had to come up with an automatic way of handling this problem.

We achieved this in the image sorting script by searching for the best local "shift" and rotation in a defined range of ± 25 pixels and 5 degrees in both directions by maximizing the overlap between the inverted enhanced image and the binary mask. This process is not always 100% correct, but it is easy to detect, when this fails, so we just ignore those few wrong results. The results of the images before and after the alignment are shown on Figure 4.

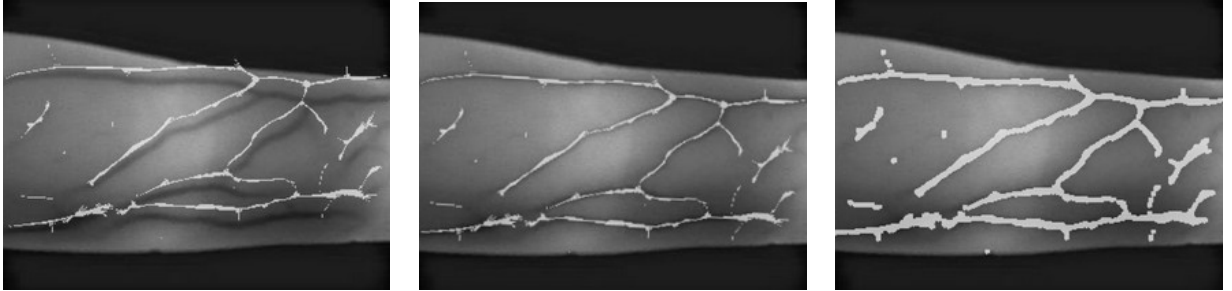


Figure 4: Before alignment (left); After alignment (middle); With dilatation applied (right)

After extracting the veins using MC for all images and methods, we evaluated their accuracy by comparing them with the baseline vein masks created in the script above. To be "less strict", we dilated the baseline mask slightly, as seen on the Figure above.

For each predicted binary mask, we computed the confusion matrix elements: true positives, false positives, false negatives, true negatives. The images were evaluated like this:

1. For each image, a mask was created to ensure that background pixels did not contribute to the results. The mask creation process went like this:
 - First the image was smoothed using a noise reduction filter based on local statistics [6].
 - The filtered image was then thresholded at its mean intensity to create initial mask.
 - Sometimes, the mask would have unfilled spots. To fix these, we detected the topmost and bottommost white pixels for every pixel column filled all the pixels between those.
 - Finally, morphological closing and dilation were applied to the mask to remove small gaps sometimes present on the sides and smooth the mask edges.

2. The Maximum Curvature extractor was applied to the image, outputting a grayscale image. The image was then binarized using Otsu's thresholding.
3. For each image, we computed TP, FP, FN and TN by comparing the predicted mask with the baseline mask.

Below on Figure 5 and Figure 6 are veins detected on the same image but different methods.

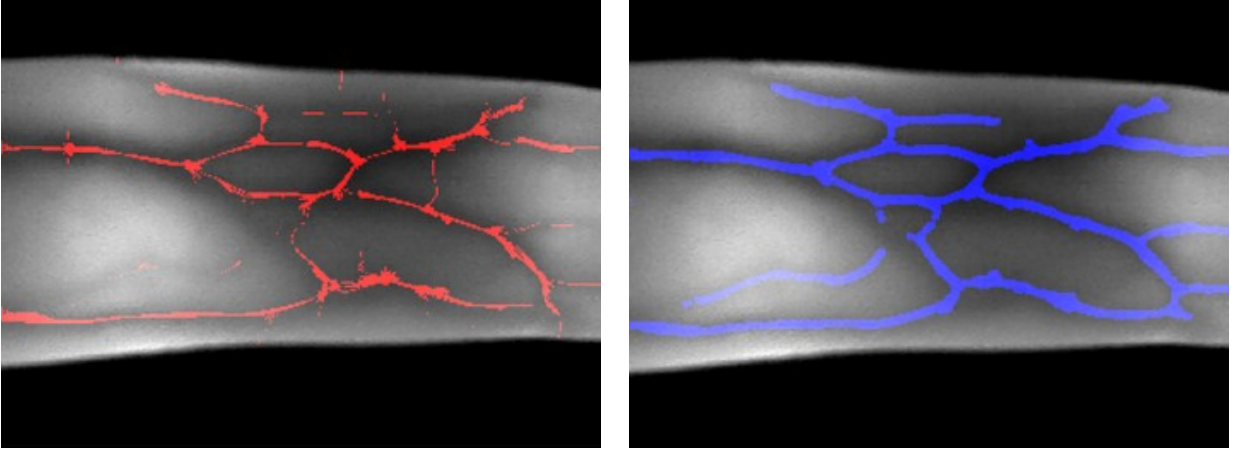


Figure 5: Veins extracted from original image (left); Baseline vein mask (right)

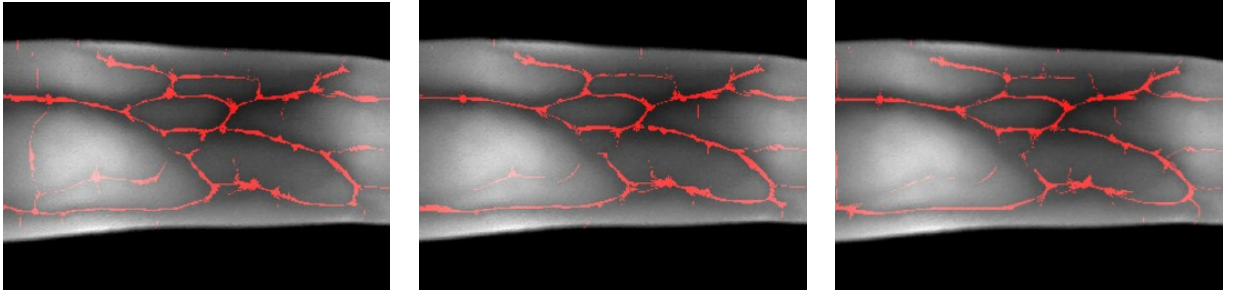


Figure 6: fixed_gabor (left); adaptive_gabor (middle); adaptive_gabor_more (right)

After calculating metrics for each image, metrics were aggregated for each method. Both the mean metrics and other metrics like accuracy, precision, recall and F1-score were computed. Because the automatic mask alignment script mentioned above did not work on 100% of the images, results with $FP > TP$ were ignored. This was an easy way of filtering out a few misaligned images to not interfere with the metrics.

After processing 500 images for each of the 3 methods as well as original images, we summarized the results in Table 4. After some testing around, we decided not to preprocess the images with an algorithm like CLAHE, because that would require extensive fine-tuning for each method respectively. Instead, we wanted to focus on using raw images without additional preprocessing. The results show that some of our preprocessing methods led to a slight improvement in the Maximum Curvature vein extraction when compared to the original images.

While the F1-scores remain relatively low due to the difficulty of pixel-wise vein extraction, images enhanced with our methods still perform better than the unprocessed ones. In particular, the *Adaptive Gabor Filters (6 filters)* approach achieved the highest F1-score and the best balance between precision and recall, slightly better than the *Fixed Gabor Filters* and the raw images. The *Adaptive Gabor Filters (10 filters)* turned out to have a slightly worse overall performance even than the raw approach.

Method	TP	FP	FN	TN	Accuracy	Precision	Recall	F1
Original	2344	487	4528	69440	0.935	0.828	0.341	0.483
Fixed Gabor Filters	2416	822	4455	69107	0.931	0.746	0.352	0.478
Adaptive Gabor Filters (6)	2482	576	4391	69350	0.935	0.812	0.361	0.500
Adaptive Gabor Filters (10)	2286	653	4586	69275	0.932	0.778	0.333	0.466

Table 1: Average and overall evaluation metrics for each method (after filtering out invalid measurements). Note: TP, FP, FN, and TN values are in pixels.

5 Conclusion

In this work, we presented a method for enhancing veins in finger images by combining convolutional neural networks with Gabor and evaluated it’s results with the Maximum Curvature method.

Our experiments demonstrated that image enhancement using neural networks can positively influence vein visibility and extraction quality. Future work could focus on using deeper or different network architectures, using larger, annotated datasets.

References

- [1] Yu Lu et al. “An Available Database for the Research of Finger Vein Recognition”. In: (2014). DOI: 10.1109/CISP.2013.6744030.
- [2] Yakun Zhang et al. “Adaptive Learning Gabor Filter for Finger-Vein Recognition”. In: *IEEE Access* 7 (2019), pp. 159821–159830. DOI: 10.1109/ACCESS.2019.2950698.
- [3] Ivan Kovač and Pavol Marák. “Finger vein recognition: utilization of adaptive gabor filters in the enhancement stage combined with SIFT/SURF-based feature extraction”. In: *Signal, Image and Video Processing* (2023). DOI: 10.1007/s11760-022-02270-8.
- [4] Victor Bros, Ketan Kotwal, and Sébastien Marcel. “Vein Enhancement with Deep Auto-Encoders to improve Finger Vein Recognition”. In: *2021 International Conference of the Biometrics Special Interest Group (BIOSIG)*. 2021, pp. 1–5. DOI: 10.1109/BIOSIG52210.2021.9548301.
- [5] Qiong Yao et al. “Visual Feature-Guided Diamond Convolutional Network for Finger Vein Recognition”. In: *Sensors* 24.18 (2024). ISSN: 1424-8220. DOI: 10.3390/s24186097. URL: <https://www.mdpi.com/1424-8220/24/18/6097>.
- [6] Jong-Sen Lee. “Digital Image Enhancement and Noise Filtering by Use of Local Statistics”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence PAMI-2.2* (1980), pp. 165–168. DOI: 10.1109/TPAMI.1980.4766994.

A Tables of adaptive Gabor filters parameters

The following tables 3 and 4 contain the difference of the Gabor parameters between two adaptive models after training. In table 2 we can see the initial configurations of the models. The initial orientation for the adaptive Gabor filter with 6 filters (table 3) had initial orientations = $[0, 18, 65, 90, 115, 160]$.

The initial orientation for the adaptive Gabor filter with 10 filters (table 4) had initial orientations = $[0, 18, 36, 54, 72, 90, 108, 126, 144, 162]$.

k_{size}	σ	λ	γ	ψ
61	5	16.0	1.000	0.000

Table 2: Initial configurations of the Gabor filter parameters.

Filter	σ	θ (deg)	λ	γ	ψ
1	4.926	13.4	15.967	1.077	-0.200
2	5.046	17.3	15.933	1.100	0.173
3	4.688	52.2	15.973	1.093	-0.122
4	4.920	69.1	15.990	1.054	0.035
5	4.742	116.7	15.897	1.170	-0.129
6	4.497	153.1	16.018	1.127	0.061

Table 3: Learned Gabor parameters for the 6-filter model.

Filter	σ	θ (deg)	λ	γ	ψ
1	5.053	6.2	15.972	1.040	0.080
2	4.946	14.3	15.948	1.098	-0.121
3	5.044	35.2	15.940	1.097	-0.144
4	4.926	47.0	15.992	1.012	0.098
5	5.024	71.4	15.939	1.110	0.004
6	4.856	85.8	15.980	1.158	0.092
7	4.919	112.1	15.974	1.063	-0.076
8	4.889	134.7	15.961	1.076	0.002
9	5.071	141.1	15.955	1.058	-0.226
10	4.946	164.2	15.934	1.104	-0.064

Table 4: Learned Gabor parameters for the 10-filter model.